

Accenture Most Asked Python DSA Questions

(Covers Arrays, Strings, Linked List, Stack, Queue, Trees, BST, DP & More)

Created by: Abhishek Rathor

Instagram: `SYNTAX ERROR (code.abhii07)`

Key Features:

- **75+ Handpicked Interview Questions** from real coding rounds & assessments.
- **Beginner to Advanced Topics:**
 - Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Dynamic Programming → Advanced Backtracking & Trie.
- **Python Implementations** with well-structured **Time & Space Complexities**.
- **Concept Explanations** and **Optimal Approaches** for quick revision.
- **Optimized Code** and patterns used in top product & service-based companies.

Why This Guide?

- ☐ **Save Time:** No need to browse scattered sources — this is your all-in-one prep resource.
- **Practice Smart:** Focused on **most frequently asked DSA problems**
- **Understand Concepts:** Each problem includes logic, patterns, dry-run approach, and edge cases.
- **Placement Ready:** Aligned with Coding Rounds (Online + Technical Interview Levels).

☐ *“Success isn’t just about writing code — it’s about solving problems effectively.”*

Arrays & Strings

1. Find the missing number in an array of 1 to N

LeetCode Problem #268

Problem Statement

You are given an array containing n distinct numbers taken from 0, 1, 2, ..., n. Find the one number that is missing from the array.

Solution Logic

Use the formula:

$$n * (n + 1) / 2$$

Subtract the actual sum from this expected sum.

Python Code

```
def missingNumber(nums):
    n = len(nums)
    total_sum = n * (n + 1) // 2
    return total_sum - sum(nums)
```

Time Complexity: O(n)

Space Complexity: O(1)

2. Move all zeroes to the end of the array

LeetCode Problem #283

Problem Statement

Move all 0's to the end while maintaining relative order.

Solution Logic

Two-pointer method to shift non-zero elements.

Python Code

```
def moveZeroes(nums):
    j = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[j] = nums[i]
            j += 1
    for k in range(j, len(nums)):
```

```
nums[k] = 0
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

3. Find the largest subarray with equal number of 0s and 1s

LeetCode Problem #525

Problem Statement

Find max length subarray with equal 0s and 1s.

Solution Logic

Replace 0 with -1, use hashmap of prefix sums.

Python Code

```
def findMaxLength(nums):
    prefix_sum = 0
    max_length = 0
    sum_index_map = {0: -1}
    for i, num in enumerate(nums):
        prefix_sum += -1 if num == 0 else 1
        if prefix_sum in sum_index_map:
            max_length = max(max_length, i - sum_index_map[prefix_sum])
        else:
            sum_index_map[prefix_sum] = i
    return max_length
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

4. Find the longest substring without repeating characters

LeetCode Problem #3

Problem Statement

Find the longest substring without duplicates.

Solution Logic

Use sliding window with a hashmap.

Python Code

```
def lengthOfLongestSubstring(s):
    char_index = {}
    left = 0
```

```

max_length = 0
for right, char in enumerate(s):
    if char in char_index and char_index[char] >= left:
        left = char_index[char] + 1
    char_index[char] = right
    max_length = max(max_length, right - left + 1)
return max_length

```

Time Complexity: $O(n)$

Space Complexity: $O(\min(m, n))$

5. Find the longest palindrome in a string

LeetCode Problem #5

Problem Statement

Return the longest palindromic substring.

Solution Logic

Use expand-around-center approach.

Python Code

```

def longestPalindrome(s):
    if not s:
        return ""

    def expandAroundCenter(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return left + 1, right - 1

    start, end = 0, 0
    for i in range(len(s)):
        l1, r1 = expandAroundCenter(i, i)
        l2, r2 = expandAroundCenter(i, i + 1)
        if r1 - l1 > end - start:
            start, end = l1, r1
        if r2 - l2 > end - start:
            start, end = l2, r2
    return s[start:end + 1]

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

6. Find the intersection of two arrays

LeetCode Problem #349

Problem Statement

Return an array of unique elements that appear in both arrays.

Solution Logic

Use set intersection.

Python Code

```
def intersection(nums1, nums2):  
    return list(set(nums1) & set(nums2))
```

Time Complexity: $O(n + m)$

Space Complexity: $O(\min(n, m))$

7. Find the union of two arrays

Modified LeetCode #349

Problem Statement

Return an array with all unique elements from both arrays.

Solution Logic

Use set union.

Python Code

```
def union(nums1, nums2):  
    return list(set(nums1) | set(nums2))
```

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

8. Find the majority element in an array

LeetCode Problem #169

Problem Statement

Return the element appearing more than $\lfloor n/2 \rfloor$ times.

Solution Logic

Use Boyer-Moore Voting Algorithm.

Python Code

```
def majorityElement(nums):
    count = 0
    candidate = None
    for num in nums:
        if count == 0:
            candidate = num
        count += (1 if num == candidate else -1)
    return candidate
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

9. Find the first non-repeating character in a string

LeetCode Problem #387

Problem Statement

Return the index of the first unique character, or -1.

Solution Logic

Count frequencies, then scan again.

Python Code

```
def firstUniqChar(s):
    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    for i, char in enumerate(s):
        if count[char] == 1:
            return i
    return -1
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

10. Find the longest common prefix in an array of strings

LeetCode Problem #14

Problem Statement

Find the longest common starting substring among an array of strings.

Solution Logic

Start with the first string, shorten prefix on mismatch.

Python Code

```
def longestCommonPrefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
        if not prefix:
            return ""
    return prefix
```

Time Complexity: $O(S)$, where S is the total number of characters

Space Complexity: $O(1)$

Linked Lists

11. Reverse a singly linked list

LeetCode Problem #206

Problem Statement

Reverse the singly linked list and return the head.

Solution Logic

Iterate and reverse `next` pointers.

Python Code

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverseList(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

12. Detect a cycle in a linked list

LeetCode Problem #141

Problem Statement

Check if a cycle exists in the list.

Solution Logic

Use Floyd's Tortoise and Hare algorithm.

Python Code

```
def hasCycle(head):
    slow, fast = head, head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

13. Merge two sorted linked lists

LeetCode Problem #21

Problem Statement

Merge and return a single sorted linked list.

Solution Logic

Use two pointers to compare elements.

Python Code

```
def mergeTwoLists(l1, l2):
    dummy = ListNode(0)
    current = dummy
    while l1 and l2:
        if l1.val < l2.val:
            current.next = l1
            l1 = l1.next
        else:
            current.next = l2
            l2 = l2.next
        current = current.next
    current.next = l1 if l1 else l2
    return dummy.next
```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

14. Find the middle of a linked list

LeetCode Problem #876

Problem Statement

Given a non-empty singly linked list, return the middle node. If there are two middle nodes, return the second one.

Solution Logic

Use two pointers: slow moves 1 step, fast moves 2 steps.

Python Code

```
def middleNode(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

15. Remove nth node from end of list

LeetCode Problem #19

Problem Statement

Remove the nth node from the end of the list.

Solution Logic

Use two pointers spaced n apart.

Python Code

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        slow = slow.next
        fast = fast.next
    slow.next = slow.next.next
```

```
return dummy.next
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

16. Remove duplicates from sorted linked list

LeetCode Problem #83

Problem Statement

Delete all duplicates in a sorted linked list.

Solution Logic

Skip nodes with same value as current.

Python Code

```
def deleteDuplicates(head):
    current = head
    while current and current.next:
        if current.val == current.next.val:
            current.next = current.next.next
        else:
            current = current.next
    return head
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

17. Add two numbers represented by linked lists

LeetCode Problem #2

Problem Statement

Add two numbers stored in linked lists.

Solution Logic

Add digits one by one, managing carry.

Python Code

```
def addTwoNumbers(l1, l2):
    dummy = ListNode(0)
    current = dummy
    carry = 0

    while l1 or l2 or carry:
```

```

        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0
        total = val1 + val2 + carry
        carry = total // 10
        current.next = ListNode(total % 10)
        current = current.next

    if l1:
        l1 = l1.next
    if l2:
        l2 = l2.next
    return dummy.next

```

Time Complexity: $O(\max(n, m))$

Space Complexity: $O(\max(n, m))$

18. Intersection of two linked lists

LeetCode Problem #160

Problem Statement

Find the intersection node of two singly linked lists.

Solution Logic

Use two pointers and switch heads when reaching the end.

Python Code

```

def getIntersectionNode(headA, headB):
    if not headA or not headB:
        return None
    a, b = headA, headB
    while a != b:
        a = a.next if a else headB
        b = b.next if b else headA
    return a

```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

19. Palindrome linked list

LeetCode Problem #234

Problem Statement

Check if a singly linked list is a palindrome.

Solution Logic

Find the middle, reverse second half, and compare.

Python Code

```
def isPalindrome(head):
    # Find middle
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    # Reverse second half
    prev = None
    while slow:
        nxt = slow.next
        slow.next = prev
        prev = slow
        slow = nxt

    # Compare halves
    left, right = head, prev
    while right:
        if left.val != right.val:
            return False
        left = left.next
        right = right.next
    return True
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

20. Flatten a multilevel doubly linked list

LeetCode Problem #430

Problem Statement

Flatten a multilevel doubly linked list into a single-level list.

Solution Logic

Use DFS or stack to handle child pointers.

Python Code

```
def flatten(head):
    if not head:
        return head
    stack = []
    current = head

    while current:
        if current.child:
            stack.append(current.next)
```

```

        if current.next:
            stack.append(current.next)
            current.next = current.child
            current.child.prev = current
            current.child = None
        if not current.next and stack:
            current.next = stack.pop()
            current.next.prev = current
        current = current.next
    return head

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

21. Swap nodes in pairs

LeetCode Problem #24

Problem Statement

Swap every two adjacent nodes.

Solution Logic

Reassign pointers while iterating.

Python Code

```

def swapPairs(head):
    dummy = ListNode(0)
    dummy.next = head
    prev = dummy

    while head and head.next:
        first = head
        second = head.next

        prev.next = second
        first.next = second.next
        second.next = first

        prev = first
        head = first.next
    return dummy.next

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

22. Remove linked list elements

LeetCode Problem #203

Problem Statement

Remove all elements with a given value.

Solution Logic

Use a dummy node to handle edge cases.

Python Code

```
def removeElements(head, val):
    dummy = ListNode(0)
    dummy.next = head
    current = dummy
    while current.next:
        if current.next.val == val:
            current.next = current.next.next
        else:
            current = current.next
    return dummy.next
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

23. Rotate list

LeetCode Problem #61

Problem Statement

Rotate the list to the right by k places.

Solution Logic

Make the list circular, then break at the right place.

Python Code

```
def rotateRight(head, k):
    if not head or k == 0:
        return head
    # Get length
    length = 1
    tail = head
    while tail.next:
        tail = tail.next
        length += 1

    k = k % length
    if k == 0:
        return head

    # Make cycle
    tail.next = head
```

```

# Find new tail
steps_to_new_tail = length - k
new_tail = head
for _ in range(steps_to_new_tail - 1):
    new_tail = new_tail.next

new_head = new_tail.next
new_tail.next = None

return new_head

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

24. Delete node in a linked list

LeetCode Problem #237

Problem Statement

Delete the given node (except tail) from a singly linked list.

Solution Logic

Copy the next node's value and pointer.

Python Code

```

def deleteNode(node):
    node.val = node.next.val
    node.next = node.next.next

```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

25. Clone a linked list with random pointer

LeetCode Problem #138

Problem Statement

Clone a linked list where each node has a random pointer.

Solution Logic

Use a hashmap to map original to copied nodes.

Python Code

```

class Node:
    def __init__(self, val, next=None, random=None):

```

```

        self.val = val
        self.next = next
        self.random = random

def copyRandomList(head):
    if not head:
        return None
    old_to_new = {}

    current = head
    while current:
        old_to_new[current] = Node(current.val)
        current = current.next

    current = head
    while current:
        old_to_new[current].next = old_to_new.get(current.next)
        old_to_new[current].random = old_to_new.get(current.random)
        current = current.next
    return old_to_new[head]

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Stacks

26. Valid Parentheses

LeetCode #20

Problem Statement

Given a string containing just the characters '(', ')', '{', '}', '[', ']', determine if the input string is valid. An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.

Solution Logic

Use a stack. Push opening brackets onto the stack. For closing brackets, check if the top of the stack matches. At the end, the stack should be empty.

Python Code

```

def isValid(s: str) -> bool:
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for ch in s:
        if ch in mapping:
            top = stack.pop() if stack else '#'
            if mapping[ch] != top:
                return False
        else:
            stack.append(ch)
    return len(stack) == 0

```

```
        stack.append(ch)
    return not stack
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

27. Min Stack

LeetCode #155

Problem Statement

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution Logic

Use two stacks:

- `stack` stores values
- `min_stack` stores the current minimum at each level

Python Code

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        val = self.stack.pop()
        if val == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

Time Complexity: $O(1)$

Space Complexity: $O(n)$

28. Evaluate Reverse Polish Notation

LeetCode #150

Problem Statement

Evaluate the value of an arithmetic expression in Reverse Polish Notation (postfix notation). Supports +, -, *, /.

Solution Logic

Use a stack:

- Push operands
- On operator, pop two operands, evaluate, and push result

Python Code

```
def evalRPN(tokens: list[str]) -> int:
    stack = []
    ops = {'+', '-', '*', '/'}
    for t in tokens:
        if t not in ops:
            stack.append(int(t))
        else:
            b = stack.pop()
            a = stack.pop()
            if t == '+':
                stack.append(a + b)
            elif t == '-':
                stack.append(a - b)
            elif t == '*':
                stack.append(a * b)
            else:
                stack.append(int(a / b)) # Truncate towards zero
    return stack[-1]
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

29. Implement Stack using Queues

LeetCode #225

Problem Statement

Implement a LIFO stack using only two queues.

Solution Logic

Use two queues. Push normally. For pop/top, move all but one to second queue, then return or remove last.

Python Code

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q1 = deque()
```

```

        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q1.append(x)

    def pop(self) -> int:
        while len(self.q1) > 1:
            self.q2.append(self.q1.popleft())
        res = self.q1.popleft()
        self.q1, self.q2 = self.q2, self.q1
        return res

    def top(self) -> int:
        while len(self.q1) > 1:
            self.q2.append(self.q1.popleft())
        res = self.q1.popleft()
        self.q2.append(res)
        self.q1, self.q2 = self.q2, self.q1
        return res

    def empty(self) -> bool:
        return not self.q1 and not self.q2

```

Time Complexity: Push: $O(1)$, Pop/Top: $O(n)$

Space Complexity: $O(n)$

30. Simplify Path

LeetCode #71

Problem Statement

Simplify a Unix-style absolute path to its canonical form.

Solution Logic

Split path by `/`, skip `.` and empty, use stack for directory parts. Pop for `...`

Python Code

```

def simplifyPath(path: str) -> str:
    stack = []
    for part in path.split('/'):
        if part == '' or part == '.':
            continue
        elif part == '..':
            if stack:
                stack.pop()
        else:
            stack.append(part)
    return '/' + '/'.join(stack)

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Queues

31. Implement Queue using Stacks

LeetCode #232

Problem Statement

Implement a FIFO queue using two stacks. Support push, pop, peek, and empty operations.

Solution Logic

Use two stacks:

- `in_stack` for enqueue (push)
 - `out_stack` for dequeue (pop) and peek
- Transfer elements from `in_stack` to `out_stack` when `out_stack` is empty.

Python Code

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        self.in_stack.append(x)

    def pop(self) -> int:
        self.peek()
        return self.out_stack.pop()

    def peek(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

    def empty(self) -> bool:
        return not self.in_stack and not self.out_stack
```

Time Complexity: Amortized $O(1)$ per operation

Space Complexity: $O(n)$

32. Sliding Window Maximum

LeetCode #239

Problem Statement

Given an integer array and window size k , find the maximum value in each sliding window.

Solution Logic

Use a deque to maintain indices in decreasing order of values. Pop smaller elements from the back, and remove elements out of window from front.

Python Code

```
from collections import deque

def maxSlidingWindow(nums: list[int], k: int) -> list[int]:
    dq = deque()
    result = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

33. Implement Circular Queue

LeetCode #622

Problem Statement

Design a circular queue with fixed capacity supporting enqueue, dequeue, front, rear, isEmpty, and isFull operations.

Solution Logic

Use a fixed-size list, a head pointer, and a count of elements. Tail is computed via modular arithmetic.

Python Code

```
class MyCircularQueue:
    def __init__(self, k: int):
        self.queue = [0] * k
        self.head = 0
        self.count = 0
        self.capacity = k

    def enqueue(self, value: int) -> bool:
        if self.count == self.capacity:
            return False
        tail = (self.head + self.count) % self.capacity
        self.queue[tail] = value
        self.count += 1
        return True

    def dequeue(self) -> bool:
```

```

        if self.count == 0:
            return False
        self.head = (self.head + 1) % self.capacity
        self.count -= 1
        return True

def Front(self) -> int:
    return self.queue[self.head] if self.count != 0 else -1

def Rear(self) -> int:
    if self.count == 0:
        return -1
    tail = (self.head + self.count - 1) % self.capacity
    return self.queue[tail]

def isEmpty(self) -> bool:
    return self.count == 0

def isFull(self) -> bool:
    return self.count == self.capacity

```

Time Complexity: $O(1)$

Space Complexity: $O(k)$

34. Number of Islands

LeetCode #200

Problem Statement

Given a 2D grid of '1's (land) and '0's (water), count the number of islands (connected components of land).

Solution Logic

Use BFS (or DFS) to traverse connected lands. Mark visited cells to avoid recounting.

Python Code (BFS)

```

from collections import deque

def numIslands(grid: list[list[str]]) -> int:
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])

    def neighbors(r, c):
        for nr, nc in [(r-1,c), (r+1,c), (r,c-1), (r,c+1)]:
            if 0 <= nr < rows and 0 <= nc < cols:
                yield nr, nc

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1

```

```

        queue = deque([(r, c)])
        grid[r][c] = '0'
        while queue:
            cr, cc = queue.popleft()
            for nr, nc in neighbors(cr, cc):
                if grid[nr][nc] == '1':
                    queue.append((nr, nc))
                    grid[nr][nc] = '0'

    return count

```

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ in worst case

35. Sliding Window for Minimum (Monotonic Queue)

(Common pattern, no direct LeetCode)

Problem Statement

Given an array and window size k , find minimum in each sliding window.

Solution Logic

Use a deque to maintain indices in increasing order. Pop larger elements from back, remove out-of-window indices from front.

Python Code

```

from collections import deque

def minSlidingWindow(nums: list[int], k: int) -> list[int]:
    dq = deque()
    result = []
    for i, val in enumerate(nums):
        while dq and nums[dq[-1]] > val:
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

Graph / BFS / DFS / Union-Find

36. Number of Provinces

LeetCode Problem #547

Problem Statement:

There are n cities. You are given an $n \times n$ matrix `isConnected` where `isConnected[i][j] = 1` means city i and city j are directly connected, and 0 otherwise. A province is a group of directly or indirectly connected cities. Return the number of provinces.

Solution Logic:

Interpret the matrix as an undirected graph adjacency matrix. Use either DFS or Union-Find to find connected components.

- **DFS:** For each city not visited, start DFS and mark all reachable cities, increment province count.
- **Union-Find:** Initially each city is its own parent; union connected pairs; count distinct parents.

```
def findCircleNum(isConnected):
    n = len(isConnected)
    visited = [False] * n

    def dfs(i):
        for j in range(n):
            if isConnected[i][j] == 1 and not visited[j]:
                visited[j] = True
                dfs(j)

    provinces = 0
    for i in range(n):
        if not visited[i]:
            visited[i] = True
            dfs(i)
            provinces += 1
    return provinces
```

Time Complexity: $O(n^2)$ — scanning adjacency matrix.

Space Complexity: $O(n)$ — visited array and recursion stack.

37. Course Schedule

LeetCode Problem #207

Problem Statement:

Given `numCourses` and prerequisites pairs `[a, b]` meaning to take course a you must first take course b , determine if you can finish all courses.

Solution Logic:

Detect cycle in directed graph using either Kahn's algorithm (BFS/topological sort) or DFS with color marking.

- **BFS (Kahn's):** Use indegree array; enqueue nodes with indegree 0; process nodes and reduce indegrees; if all nodes processed, no cycle.
- **DFS:** Mark nodes as unvisited, visiting, visited; detect back edges.

```
from collections import deque

def canFinish(numCourses, prerequisites):
    adj = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses
    for a, b in prerequisites:
        adj[b].append(a)
        indegree[a] += 1

    q = deque([i for i in range(numCourses) if indegree[i] == 0])
    count = 0
    while q:
        node = q.popleft()
        count += 1
        for nei in adj[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                q.append(nei)
    return count == numCourses
```

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

38. Clone Graph

LeetCode Problem #133

Problem Statement:

Given a reference to a node in a connected undirected graph, return a deep copy (clone) of the graph.

Solution Logic:

Use DFS or BFS to traverse original graph. Keep a map from original node to cloned node. When visiting a node, clone it if not cloned, then recursively clone neighbors.

```
def cloneGraph(node):
    if not node:
        return None
    old_to_new = {}

    def dfs(n):
        if n in old_to_new:
            return old_to_new[n]
        copy = Node(n.val)
        old_to_new[n] = copy
        for nei in n.neighbors:
            copy.neighbors.append(dfs(nei))
```

```

        copy.neighbors.append(dfs(nei))
    return copy

return dfs(node)

```

Time Complexity: $O(N + M)$ where N is nodes and M is edges.

Space Complexity: $O(N)$

39. Word Ladder (Shortest Transformation Sequence)

LeetCode Problem #127

Problem Statement:

Given `beginWord`, `endWord`, and a list `wordList`, find the length of the shortest transformation sequence from `beginWord` to `endWord` where only one letter can change at a time and all intermediate words must be in `wordList`.

Solution Logic:

Model words as nodes and edges between words differing by one letter. Use BFS with preprocessing of generic intermediate patterns (replace one letter with `*`) to find neighbors efficiently.

```

from collections import deque, defaultdict

def ladderLength(beginWord, endWord, wordList):
    if endWord not in wordList:
        return 0
    L = len(beginWord)
    all_combo = defaultdict(list)
    for w in wordList:
        for i in range(L):
            pattern = w[:i] + '*' + w[i+1:]
            all_combo[pattern].append(w)

    q = deque([(beginWord, 1)])
    visited = {beginWord}
    while q:
        word, level = q.popleft()
        for i in range(L):
            pattern = word[:i] + '*' + word[i+1:]
            for neighbor in all_combo[pattern]:
                if neighbor == endWord:
                    return level + 1
                if neighbor not in visited:
                    visited.add(neighbor)
                    q.append((neighbor, level + 1))
            all_combo[pattern] = []
    return 0

```

Time Complexity: $O(M \times N \times 26)$ — M word length, N number of words.

Space Complexity: $O(M \times N)$

40. Number of Islands II

LeetCode Problem #305

Problem Statement:

Given m , n and a list of positions where land is added sequentially in an all-water grid, return the number of islands after each addition.

Solution Logic:

Use Union-Find to dynamically union newly added land with adjacent lands, maintaining count of distinct islands.

```
def numIslands2(m, n, positions):
    parent = {}
    rank = {}
    count = 0
    dirs = [(1,0), (-1,0), (0,1), (0,-1)]

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
        nonlocal count
        ra, rb = find(a), find(b)
        if ra == rb:
            return
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra
            rank[ra] += 1
        count -= 1

    res = []
    for r, c in positions:
        if (r, c) in parent:
            res.append(count)
            continue
        parent[(r, c)] = (r, c)
        rank[(r, c)] = 0
        count += 1
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if (nr, nc) in parent:
                union((r, c), (nr, nc))
        res.append(count)
    return res
```

Time Complexity: Amortized $\sim O(K \times \alpha(mn))$

Space Complexity: $O(m \times n)$

41. Number of Connected Components in an Undirected Graph

LeetCode Problem #323

Problem Statement:

Given n nodes and edges, return the number of connected components.

Solution Logic:

Use Union-Find or DFS. Union connected nodes, count distinct roots.

```
def countComponents(n, edges):
    parent = list(range(n))
    rank = [0] * n

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra
            rank[ra] += 1

    for u, v in edges:
        union(u, v)

    roots = set(find(i) for i in range(n))
    return len(roots)
```

Time Complexity: $\sim O((n + E) \times \alpha(n))$

Space Complexity: $O(n)$

42. Course Schedule II

LeetCode Problem #210

Problem Statement:

Same as Course Schedule but return one possible order to finish all courses or empty list if impossible.

Solution Logic:

Use topological sort (Kahn's BFS). Build order list if no cycle detected.

```
from collections import deque

def findOrder(numCourses, prerequisites):
    adj = [[] for _ in range(numCourses)]
```

```

indegree = [0] * numCourses
for a, b in prerequisites:
    adj[b].append(a)
    indegree[a] += 1

q = deque([i for i in range(numCourses) if indegree[i] == 0])
order = []
while q:
    node = q.popleft()
    order.append(node)
    for nei in adj[node]:
        indegree[nei] -= 1
        if indegree[nei] == 0:
            q.append(nei)
return order if len(order) == numCourses else []

```

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

43. Find Redundant Connection

LeetCode Problem #684

Problem Statement

In a graph that started as a tree with n nodes, one extra edge is added, making it have a cycle. Given edges, find the edge that can be removed so that the resulting graph is a tree of n nodes. If multiple answers, return the one that appears last.

Solution Logic

Use Union-Find. Iterate edges; for each edge (u, v) , check if u and v already share the same root; if yes, that edge is redundant. Otherwise union them.

Python Code

```

def findRedundantConnection(edges):
    n = len(edges)
    parent = list(range(n+1))
    rank = [0] * (n+1)

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra

```

```

        rank[ra] += 1
    return True

for u, v in edges:
    if not union(u, v):
        return [u, v]
return []

```

Time Complexity

$\sim O(E \times \alpha(n))$ (inverse Ackermann function, almost constant)

Space Complexity

$O(n)$

44. Surrounded Regions

LeetCode Problem #130

Problem Statement

Given an $m \times n$ board containing 'X' and 'O', capture all regions surrounded by 'X'. A region is captured by flipping all 'O's into 'X's if they are fully surrounded (no path to border).

Solution Logic

Use DFS (or BFS) from border 'O's to mark safe regions temporarily (e.g., mark as '#'). Then flip all other 'O's to 'X' and revert '#' back to 'O'.

Python Code

```

def solve(board):
    if not board or not board[0]:
        return
    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != 'O':
            return
        board[r][c] = '#'
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            dfs(r + dr, c + dc)

    # DFS from border 'O's
    for r in range(rows):
        for c in [0, cols-1]:
            if board[r][c] == 'O':
                dfs(r, c)
    for c in range(cols):
        for r in [0, rows-1]:
            if board[r][c] == 'O':
                dfs(r, c)

    # Flip internal 'O' to 'X' and '#' back to 'O'
    for r in range(rows):
        for c in range(cols):

```

```

        if board[r][c] == 'O':
            board[r][c] = 'X'
        elif board[r][c] == '#':
            board[r][c] = 'O'

```

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$ for recursion stack in worst case

45. Pacific Atlantic Water Flow

LeetCode Problem #417

Problem Statement

Given an $m \times n$ height matrix, water can flow from a cell to its neighbor if neighbor's height $\leq$ current. Find all coordinates where water can flow to both the Pacific (top/left edges) and Atlantic (bottom/right edges).

Solution Logic

Reverse the flow: start DFS/BFS from the borders of Pacific and Atlantic, marking cells reachable going "uphill". The intersection of reachable cells is the answer.

Python Code

```

def pacificAtlantic(heights):
    if not heights or not heights[0]:
        return []
    m, n = len(heights), len(heights[0])
    pac = [[False]*n for _ in range(m)]
    atl = [[False]*n for _ in range(m)]

    def dfs(r, c, visited, prev_height):
        if (r < 0 or r >= m or c < 0 or c >= n or
            visited[r][c] or heights[r][c] < prev_height):
            return
        visited[r][c] = True
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            dfs(r + dr, c + dc, visited, heights[r][c])

    for i in range(m):
        dfs(i, 0, pac, heights[i][0])
        dfs(i, n-1, atl, heights[i][n-1])
    for j in range(n):
        dfs(0, j, pac, heights[0][j])
        dfs(m-1, j, atl, heights[m-1][j])

    res = []
    for i in range(m):
        for j in range(n):
            if pac[i][j] and atl[i][j]:
                res.append([i, j])

```

```
return res
```

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

46. Maximum Depth of Binary Tree

LeetCode Problem #104**Problem Statement**

Given the root of a binary tree, return its maximum depth (number of nodes along the longest path from root to leaf).

Solution Logic

Use DFS recursion. $\text{Depth} = 1 + \max(\text{depth left subtree}, \text{depth right subtree})$.

Python Code

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def maxDepth(root):
    if not root:
        return 0
    left_depth = maxDepth(root.left)
    right_depth = maxDepth(root.right)
    return 1 + max(left_depth, right_depth)
```

Time Complexity

$O(n)$

Space Complexity

$O(h)$, h = tree height (recursion stack)

47. Validate Binary Search Tree

LeetCode Problem #98**Problem Statement**

Given a binary tree root, determine if it is a valid BST (left subtree < node < right subtree).

Solution Logic

Use recursion with min/max bounds. Check if node.val is within bounds; recurse left with upper bound = node.val and right with lower bound = node.val.

Python Code

```
def isValidBST(root, lower=float('-inf'), upper=float('inf')):
    if not root:
        return True
    val = root.val
    if val <= lower or val >= upper:
        return False
    if not isValidBST(root.left, lower, val):
        return False
    if not isValidBST(root.right, val, upper):
        return False
    return True
```

Time Complexity

$O(n)$

Space Complexity

$O(h)$

48. Inorder Traversal (Recursive & Iterative)

LeetCode Problem #94

Problem Statement

Given root of a binary tree, return its inorder traversal (left, node, right).

Solution Logic

- Recursive: straightforward DFS left-node-right
- Iterative: use stack, push all left nodes, then process node and go right

Python Code (Iterative)

```
def inorderTraversal(root):
    res = []
    stack = []
    current = root
    while current or stack:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        res.append(current.val)
        current = current.right
    return res
```

Time Complexity

$O(n)$

Space Complexity

$O(h)$

49. Preorder & Postorder Traversals**LeetCode Problems #144 (Preorder), #145 (Postorder)****Problem Statement**

Return preorder (node, left, right) or postorder (left, right, node) traversal of a binary tree.

Solution Logic

- Preorder: push root, then right, then left to stack
- Postorder: reverse modified preorder or use two stacks

Python Code (Preorder)

```
def preorderTraversal(root):
    if not root:
        return []
    res = []
    stack = [root]
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return res
```

Time Complexity

$O(n)$

Space Complexity

$O(h)$

50. Level Order Traversal / Breadth-First**LeetCode Problem #102****Problem Statement**

Given the root of a binary tree, return its level order traversal (i.e., values from left to right level by level).

Solution Logic

Use a queue (BFS). For each level, dequeue all nodes at that level, record values, and enqueue their children.

Python Code

```
from collections import deque

def levelOrder(root):
    if not root:
        return []

    res = []
    queue = deque([root])

    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        res.append(level)

    return res
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

51. Lowest Common Ancestor of a Binary Tree

LeetCode Problem #236

Problem Statement

Given a binary tree and two nodes p and q , find their lowest common ancestor (LCA).

Solution Logic

Use DFS.

- If root is None, return None
- If root is p or q , return root
- Recurse left and right.
- If both sides return a node, root is LCA.

Python Code

```
def lowestCommonAncestor(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
```

```
right = lowestCommonAncestor(root.right, p, q)
if left and right:
    return root
return left or right
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ (recursion stack)

52. Construct Binary Tree from Preorder and Inorder

LeetCode Problem #105

Problem Statement

Given preorder and inorder traversal of a binary tree, construct the original tree.

Solution Logic

- First node in preorder is root.
- Use hashmap to find root index in inorder.
- Recursively build left and right subtrees using sliced lists or indices.

Python Code

```
def buildTree(preorder, inorder):
    inorder_index = {val: i for i, val in enumerate(inorder)}

    def helper(pre_left, pre_right, in_left, in_right):
        if pre_left > pre_right:
            return None
        root_val = preorder[pre_left]
        root = TreeNode(root_val)
        index = inorder_index[root_val]
        left_size = index - in_left

        root.left = helper(pre_left + 1, pre_left + left_size, in_left, index - 1)
        root.right = helper(pre_left + left_size + 1, pre_right, index + 1,
                           in_right)
        return root

    return helper(0, len(preorder) - 1, 0, len(inorder) - 1)
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

53. Serialize and Deserialize Binary Tree

LeetCode Problem #297

Problem Statement

Design an algorithm to serialize and deserialize a binary tree into/from a string.

Solution Logic

Use pre-order traversal. For nulls, use sentinel (e.g., "#").

- Serialize: build string using DFS.
- Deserialize: split string and reconstruct recursively.

Python Code

```
class Codec:
    def serialize(self, root):
        def dfs(node):
            if not node:
                vals.append("#")
                return
            vals.append(str(node.val))
            dfs(node.left)
            dfs(node.right)

        vals = []
        dfs(root)
        return ','.join(vals)

    def deserialize(self, data):
        def dfs():
            val = next(vals)
            if val == "#":
                return None
            node = TreeNode(int(val))
            node.left = dfs()
            node.right = dfs()
            return node

        vals = iter(data.split(','))
        return dfs()
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

54. Binary Tree Maximum Path Sum

LeetCode Problem #124

Problem Statement

Find the maximum path sum in a binary tree. A path can start and end at any node.

Solution Logic

Use DFS. At each node, compute:

- max gain from left and right child
 - max path through node = node + left_gain + right_gain
- Keep a global max value updated. Return max gain to parent.

Python Code

```
def maxPathSum(root):
    max_sum = float('-inf')

    def dfs(node):
        nonlocal max_sum
        if not node:
            return 0
        left = max(dfs(node.left), 0)
        right = max(dfs(node.right), 0)
        max_sum = max(max_sum, node.val + left + right)
        return node.val + max(left, right)

    dfs(root)
    return max_sum
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

55. Diameter of Binary Tree

LeetCode Problem #543

Problem Statement

The diameter of a binary tree is the length of the longest path between any two nodes (can pass through root or not).

Solution Logic

Use DFS. At each node, compute depth of left and right.

- Diameter through node = left_depth + right_depth
Keep a global max diameter.

Python Code

```
def diameterOfBinaryTree(root):
    max_diameter = 0

    def dfs(node):
        nonlocal max_diameter
        if not node:
            return 0
        left = dfs(node.left)
        right = dfs(node.right)
        max_diameter = max(max_diameter, left + right)
        return 1 + max(left, right)

    dfs(root)
    return max_diameter
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

Heaps / Greedy / Priority Queue, Backtracking

56. Merge k Sorted Lists

LeetCode Problem #23

Problem Statement

You are given an array of k linked lists, each sorted in ascending order. Merge all the lists into one sorted list and return it.

Solution Logic

Use a min-heap (priority queue) to always pick the smallest current node across all lists.

- Push the head of each list into the heap
- Pop the smallest, append to result, and push its next (if exists)

Python Code

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

    # for comparing nodes in heap
    def __lt__(self, other):
        return self.val < other.val

def mergeKLists(lists):
    min_heap = []
    for ln in lists:
        if ln:
            heapq.heappush(min_heap, ln)
    dummy = ListNode(0)
    tail = dummy
    while min_heap:
        node = heapq.heappop(min_heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(min_heap, node.next)
    return dummy.next
```

Time Complexity: $O(N \log k)$ — N = total nodes, k = number of lists

Space Complexity: $O(k)$ — heap size

57. Task Scheduler

LeetCode Problem #621

Problem Statement

Given a list of tasks (A-Z) and a cooldown n , find the least time units needed to finish all tasks with cooldowns between identical tasks.

Solution Logic

Greedy approach using max frequency:

- Count frequencies
- Use formula: $(\text{max_count} - 1) * (n + 1) + \text{num_max}$
- Final answer is max of this formula and total number of tasks

Python Code

```
from collections import Counter

def leastInterval(tasks, n):
    freq = Counter(tasks)
    max_count = max(freq.values())
    num_max = sum(1 for v in freq.values() if v == max_count)
    part_count = max_count - 1
    part_length = n + 1
    empty_slots = part_count * part_length + num_max
    return max(empty_slots, len(tasks))
```

Time Complexity: $O(T + k)$ — T = total tasks, k = unique tasks

Space Complexity: $O(k)$

58. Permutations (Backtracking)

LeetCode Problem #46

Problem Statement

Given a list of distinct integers `nums`, return all possible permutations.

Solution Logic

Use backtracking (DFS):

- At each level, pick unused numbers
- Track path and used elements
- Backtrack after recursion

Python Code

```
def permute(nums):
    res = []
    path = []
    used = [False] * len(nums)

    def dfs():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if not used[i]:
                path.append(nums[i])
                used[i] = True
                dfs()
                path.pop()
                used[i] = False
```

```

        if len(path) == len(nums):
            res.append(path.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            dfs()
            path.pop()
            used[i] = False

    dfs()
    return res

```

Time Complexity: $O(n!)$

Space Complexity: $O(n)$

59. Combination Sum

LeetCode Problem #39

Problem Statement

Given an array of candidates and a target, return all combinations where the chosen numbers sum to the target. Reuse of elements is allowed.

Solution Logic

Backtracking + DFS:

- Sort candidates
- At each level, choose current or skip
- Reuse allowed → stay at current index

Python Code

```

def combinationSum(candidates, target):
    res = []
    candidates.sort()
    path = []

    def dfs(start, remain):
        if remain == 0:
            res.append(path.copy())
            return
        if remain < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            dfs(i, remain - candidates[i]) # not i + 1 because reuse allowed
            path.pop()

    dfs(0, target)
    return res

```

Time Complexity: Exponential (depends on number of combinations)

Space Complexity: $O(\text{target})$ — depth of recursion stack

60. Word Search

LeetCode Problem #79

Problem Statement

Given a 2D grid and a word, check if the word exists in the grid. You may move in 4 directions, but cannot reuse the same cell.

Solution Logic

DFS from each cell:

- Check character match
- Mark as visited (e.g., '#')
- Recurse in 4 directions
- Unmark on backtrack

Python Code

```
def exist(board, word):
    m, n = len(board), len(board[0])

    def dfs(r, c, idx):
        if idx == len(word):
            return True
        if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != word[idx]:
            return False
        temp = board[r][c]
        board[r][c] = '#'
        found = (dfs(r+1, c, idx+1) or
                 dfs(r-1, c, idx+1) or
                 dfs(r, c+1, idx+1) or
                 dfs(r, c-1, idx+1))
        board[r][c] = temp
        return found

    for i in range(m):
        for j in range(n):
            if dfs(i, j, 0):
                return True
    return False
```

Time Complexity: $O(m * n * 4^L)$, where L = word length

Space Complexity: $O(L)$ recursion depth

61. Fibonacci with Memoization (DP / Recursion + Memo)

LeetCode Problem #509

Problem Statement

Compute the n th Fibonacci number, where $\text{fib}(0) = 0$, $\text{fib}(1) = 1$.

Solution Logic

Use recursion + memoization (top-down DP). Store computed values to avoid redundant computation.

Python Code

```
def fib(n, memo = {}):
    if n in memo:
        return memo[n]
    if n < 2:
        return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$ — recursion + memo

62. Climbing Stairs (DP)

LeetCode Problem #70

Problem Statement

You are climbing a staircase with n steps. You can take 1 or 2 steps at a time. How many distinct ways to climb to the top?

Solution Logic

Classic DP: Similar to Fibonacci. $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$

Python Code

```
def climbStairs(n):
    if n <= 1:
        return 1
    dp0, dp1 = 1, 1
    for i in range(2, n+1):
        dp0, dp1 = dp1, dp0 + dp1
    return dp1
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

63. Minimum Path Sum

LeetCode Problem #64

Problem Statement

Given a $m \times n$ grid filled with non-negative numbers, find a path from top-left to bottom-right minimizing the sum of values along the path (only right/down moves allowed).

Solution Logic

DP grid: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

Python Code

```
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    for i in range(m):
        for j in range(n):
            if i == 0 and j == 0:
                continue
            elif i == 0:
                grid[i][j] += grid[i][j-1]
            elif j == 0:
                grid[i][j] += grid[i-1][j]
            else:
                grid[i][j] += min(grid[i-1][j], grid[i][j-1])
    return grid[m-1][n-1]
```

Time Complexity: $O(m \times n)$

Space Complexity: $O(1)$ if modifying input, $O(m \times n)$ otherwise

64. Longest Increasing Subsequence (LIS)

LeetCode Problem #300

Problem Statement

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Solution Logic

Greedy + Binary Search: Maintain a `tails` array where each element represents the minimum tail for subsequences of that length.

Python Code

```
import bisect

def lengthOfLIS(nums):
    tails = []
    for x in nums:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
```

```
        tails[i] = x
    return len(tails)
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

65. Coin Change (Unbounded Knapsack)

LeetCode Problem #322

Problem Statement

Given coins of various denominations and a total amount, return the fewest coins needed to make that amount. Return -1 if not possible.

Solution Logic

1D DP: For each coin, update $dp[amount] = \min(dp[amount], dp[amount - coin] + 1)$

Python Code

```
def coinChange(coins, amount):
    MAX = float('inf')
    dp = [0] + [MAX] * amount
    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != MAX else -1
```

Time Complexity: $O(\text{amount} \times \text{len}(\text{coins}))$

Space Complexity: $O(\text{amount})$

66. Longest Palindromic Subsequence

LeetCode Problem #516

Problem Statement

Given a string s , return the length of its longest palindromic subsequence.

Solution Logic

2D DP:

- If $s[i] == s[j]$, then $dp[i][j] = dp[i+1][j-1] + 2$
- Else: $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$

Python Code

```
def longestPalindromeSubseq(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
```

```

for i in range(n):
    dp[i][i] = 1
for length in range(2, n+1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j]:
            dp[i][j] = dp[i+1][j-1] + 2 if i+1 <= j-1 else 2
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1])
return dp[0][n-1]

```

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

67. Edit Distance (Levenshtein Distance)

LeetCode Problem #72

Problem Statement

Given two strings, return the minimum number of insertions, deletions, and replacements required to convert one string into the other.

Solution Logic

DP 2D:

- If chars match: $dp[i][j] = dp[i-1][j-1]$
- Else: $1 + \min(\text{insert}, \text{delete}, \text{replace})$

Python Code

```

def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

```

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

68. Decode Ways

LeetCode Problem #91

Problem Statement

Given a digit string, return the number of ways to decode it using mapping '1' → 'A', ..., '26' → 'Z'.

Solution Logic

DP:

- If last digit is valid (1–9), add $dp[i-1]$
- If last two digits are 10–26, add $dp[i-2]$

Python Code

```
def numDecodings(s):
    n = len(s)
    dp = [0] * (n + 1)
    dp[0] = 1
    dp[1] = 0 if s[0] == '0' else 1
    for i in range(2, n + 1):
        one = int(s[i-1:i])
        two = int(s[i-2:i])
        if 1 <= one <= 9:
            dp[i] += dp[i-1]
        if 10 <= two <= 26:
            dp[i] += dp[i-2]
    return dp[n]
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

69. Longest Increasing Path in a Matrix

LeetCode Problem #329

Problem Statement

Given an $m \times n$ matrix, return the length of the longest increasing path. You can move in four directions (up, down, left, right), and you may not revisit a cell in the same path.

Solution Logic

Use DFS + memoization. For each cell, compute the longest path starting from that cell. Store intermediate results in a memo table to avoid recomputation.

Python Code

```
def longestIncreasingPath(matrix):
    if not matrix or not matrix[0]:
        return 0
    m, n = len(matrix), len(matrix[0])
    memo = [[0]*n for _ in range(m)]
```

```

def dfs(r, c):
    if memo[r][c]:
        return memo[r][c]
    best = 1
    for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < m and 0 <= nc < n and matrix[nr][nc] > matrix[r][c]:
            best = max(best, 1 + dfs(nr, nc))
    memo[r][c] = best
    return best

ans = 0
for i in range(m):
    for j in range(n):
        ans = max(ans, dfs(i, j))
return ans

```

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

70. Word Break

LeetCode Problem #139

Problem Statement

Given a string s and a dictionary `wordDict`, return `True` if s can be segmented into a space-separated sequence of dictionary words.

Solution Logic

Use DP: $dp[i] = \text{True}$ if $s[:i]$ can be segmented. Try every possible partition ending at i .

Python Code

```

def wordBreak(s, wordDict):
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    wordSet = set(wordDict)

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True
                break
    return dp[n]

```

Time Complexity: $O(n^2 \times k)$ where k is average lookup time in the set

Space Complexity: $O(n)$

71. Palindrome Partitioning

LeetCode Problem #131

Problem Statement

Given a string s , partition it such that every substring is a palindrome. Return all such possible palindrome partitionings.

Solution Logic

Backtracking + precomputed DP table to quickly check if substrings are palindromes.

Python Code

```
def partition(s):
    res = []
    n = len(s)
    dp = [[False]*n for _ in range(n)]
    for i in range(n-1, -1, -1):
        for j in range(i, n):
            dp[i][j] = (s[i] == s[j]) and (j - i < 2 or dp[i+1][j-1])

    path = []
    def dfs(start):
        if start == n:
            res.append(path.copy())
            return
        for end in range(start, n):
            if dp[start][end]:
                path.append(s[start:end+1])
                dfs(end+1)
                path.pop()

    dfs(0)
    return res
```

Time Complexity: Exponential (due to all possible partitions)

Space Complexity: $O(n^2)$ for dp + recursion stack

72. Minimum Window Substring

LeetCode Problem #76

Problem Statement

Given two strings s and t , return the minimum window in s that contains all characters of t . If no such window exists, return "".

Solution Logic

Sliding window with character counts. Expand to right to include all characters, then contract from left to find minimum.

Python Code

```

from collections import Counter

def minWindow(s, t):
    if not t or not s:
        return ""
    dict_t = Counter(t)
    required = len(dict_t)
    l, r = 0, 0
    formed = 0
    window_counts = {}
    ans = float('inf'), None, None

    while r < len(s):
        c = s[r]
        window_counts[c] = window_counts.get(c, 0) + 1
        if c in dict_t and window_counts[c] == dict_t[c]:
            formed += 1
        while l <= r and formed == required:
            c = s[l]
            if r - l + 1 < ans[0]:
                ans = (r - l + 1, l, r)
            window_counts[c] -= 1
            if c in dict_t and window_counts[c] < dict_t[c]:
                formed -= 1
            l += 1
        r += 1

    return "" if ans[0] == float('inf') else s[ans[1]:ans[2]+1]

```

Time Complexity: $O(|S| + |T|)$

Space Complexity: $O(|S| + |T|)$

73. Word Search II

LeetCode Problem #212

Problem Statement

Given a 2D board and a list of words, return all words found in the board. Each word must be constructed from adjacent letters without reusing a cell.

Solution Logic

Use a Trie for word lookup + backtracking DFS for exploring the board.

Python Code

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None

def buildTrie(words):
    root = TrieNode()
    for word in words:
        node = root

```

```

        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.word = word
        return root

def findWords(board, words):
    m, n = len(board), len(board[0])
    root = buildTrie(words)
    res = []

    def dfs(r, c, node):
        char = board[r][c]
        if char not in node.children:
            return
        next_node = node.children[char]
        if next_node.word:
            res.append(next_node.word)
            next_node.word = None
        board[r][c] = '#'
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < m and 0 <= nc < n:
                dfs(nr, nc, next_node)
        board[r][c] = char

    for i in range(m):
        for j in range(n):
            dfs(i, j, root)
    return res

```

Time Complexity: $O(m \times n \times 4^L)$, where L is max word length

Space Complexity: $O(\text{sum of word lengths} + \text{recursion stack})$

74. Burst Balloons

LeetCode Problem #312

Problem Statement

Given n balloons, each with a number, burst them to maximize coins collected. Bursting balloon i gives coins $\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$.

Solution Logic

DP on intervals: choose k as last balloon to burst in range (i, j) and solve recursively.

Python Code

```

def maxCoins(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0]*n for _ in range(n)]

    for length in range(2, n):
        for left in range(n - length):
            right = left + length

```

```

        for k in range(left + 1, right):
            dp[left][right] = max(dp[left][right],
                                   dp[left][k] + nums[left]*nums[k]*nums[right] + dp[k][right])
    return dp[0][n-1]

```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

75. Regular Expression Matching

LeetCode Problem #10

Problem Statement

Implement regular expression matching with support for `.` (any char) and `*` (zero or more of previous char).

Solution Logic

$DP[i][j]$ = does $s[:i]$ match $p[:j]$

- If $p[j-1] == '*'$: use zero occurrence or use it if previous char matches
- Else: check char match and previous DP

Python Code

```

def isMatch(s, p):
    m, n = len(s), len(p)
    dp = [[False]*(n+1) for _ in range(m+1)]
    dp[0][0] = True

    for j in range(2, n+1):
        if p[j-1] == '*':
            dp[0][j] = dp[0][j-2]

    def matches(i, j):
        if i == 0:
            return False
        if p[j-1] == '.':
            return True
        return s[i-1] == p[j-1]

    for i in range(1, m+1):
        for j in range(1, n+1):
            if p[j-1] == '*':
                dp[i][j] = dp[i][j-2] or (matches(i, j-1) and dp[i-1][j])
            else:
                if matches(i, j):
                    dp[i][j] = dp[i-1][j-1]

    return dp[m][n]

```

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

Congratulations! You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills. Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: [@SYNTAXERROR](#). Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved. This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.